

urdu-slm: Deep Dive

A from-scratch Urdu language model, explained end to end

Explainer generated from the project source and README

Scope and honesty statement

This document describes exactly what exists in the **urdu-slm** repository today, verified by reading every source file. The repository is explicitly labelled **phase 1**: a complete, verified data-to-model pipeline plus a tiny (13M parameter) proof-of-concept model trained on a CPU. **Phase 2** — training the real $\sim 124\text{M}$ -parameter **base** model on a GPU — has *not* been run. Nothing in this document claims a trained base model, its perplexity, or its generation quality, because those do not exist yet. Every number quoted below (corpus sizes, token counts, loss values, test counts) is copied from the project’s own README, its JSON stat files, or its source code.

Contents

1	What this project is	2
1.1	High-level pipeline	3
1.2	What is in the repository	3
2	The data pipeline (pipeline/)	4
2.1	Data sources	4
2.2	Reading raw formats (<code>sources.py</code>)	4
2.3	Normalization (<code>normalize.py</code>)	4
2.4	Exact and near-duplicate removal (<code>dedup.py</code>)	5
2.5	Corpus assembly and split (<code>build_corpus.py</code> , <code>common.py</code>)	7
2.6	Measured corpus statistics (real numbers, from <code>data/corpus_stats.json</code>)	7
3	The tokenizer	8
3.1	Measured compression against GPT-2’s tokenizer	8
4	The model (model/)	9
4.1	Overall shape: a pre-norm decoder stack	9
4.2	RMSNorm	9
4.3	Rotary position embeddings (RoPE)	9
4.4	Attention, and grouped-query attention	10
4.5	SwiGLU feed-forward layer	11
4.6	Tied embeddings	11
4.7	Initialization	11
4.8	Size presets	11
5	Training (<code>train.py</code>)	13

5.1	Token-count scheduling, not step-count scheduling	13
5.2	Gradient accumulation	13
5.3	Mixed precision (AMP)	13
5.4	Cosine learning-rate schedule with warmup	13
5.5	Gradient clipping	14
5.6	Resumability	14
5.7	Single-GPU and multi-GPU (DDP) paths	14
6	Evaluation (eval/)	15
6.1	Held-out perplexity (<code>perplexity.py</code>)	15
6.2	Script-mix diagnostic (<code>script_mix.py</code>)	15
6.3	Few-shot cloze probe (<code>cloze.py</code> , <code>evaldata/</code>)	15
6.4	Tokenizer compression comparison (<code>tokenizer_compare.py</code>)	16
6.5	Loss-curve plotting (<code>plot_loss.py</code>)	16
7	The tiny proof run (executed here, on CPU)	17
8	Testing (tests/)	18
8.1	Model tests (<code>tests/test_model.py</code>)	18
8.2	Pipeline tests (<code>tests/test_pipeline.py</code>)	18
9	Status: phase 1 done, phase 2 not yet run	19
9.1	What is actually done	19
9.2	The token-budget argument (Chinchilla-style), as written in <code>docs/GPU_RUN.md</code>	19
9.3	Cost estimate for the planned run (documented, not yet spent)	19
10	What the author learned and would do differently	20
11	License	21

1 What this project is

`urdu-slm` is a language model for Urdu (a Perso-Arabic-script, right-to-left language spoken mainly in Pakistan and India) built entirely from scratch: no pretrained tokenizer, no modeling code borrowed from the `transformers` library. The author’s stated motivation is that most hobby “train a GPT” projects use English data and an English-trained tokenizer, which quietly sidesteps every hard problem a low-resource, non-Latin-script language actually poses:

- Text needs **script normalization** (the same Urdu letter can be encoded several different ways in Unicode).
- A generic tokenizer (like GPT-2’s) **shreds** Urdu text into many more tokens than necessary, because it was never trained on Urdu.
- Filtering “is this really Urdu?” needs a **language-identification** step, because raw web/Wikipedia dumps mix in English boilerplate.
- The available clean Urdu text is **small**, which forces an honest conversation about how much model that data can actually support.

Definition: Language model (LM)

A statistical model that, given some text, predicts what token (word-piece) comes next. Trained repeatedly on next-token prediction over a huge amount of text, it learns grammar, facts, and style well enough to generate plausible continuations of a prompt.

Definition: Decoder-only transformer

A neural network architecture (introduced in “Attention Is All You Need”, 2017) built from stacked blocks that each mix information across positions in a sequence (*self-attention*) and then transform each position independently (a small feed-forward network, the *MLP*). “Decoder-only” means it can only look at earlier tokens when predicting the next one (this is called *causal* or *autoregressive*), which is exactly what GPT-style models do.

1.1 High-level pipeline

Figure 1 shows the whole system: raw public text goes in one end, a trained checkpoint and evaluation reports come out the other.

1.2 What is in the repository

- **Data pipeline** (`pipeline/`): streaming, resumable stages that turn raw downloads into tokenized training shards.
- **Tokenizer**: a 32,000-token byte-level BPE trained on the cleaned corpus, with a measured compression comparison against GPT-2’s tokenizer.
- **Model** (`model/`): a decoder-only transformer with RMSNorm, rotary position embeddings, SwiGLU feed-forward layers, tied input/output embeddings, and attention that supports grouped-query attention. Three size presets: `tiny`, `small`, `base`.
- **Training** (`train.py`): mixed precision, gradient accumulation, cosine learning-rate schedule with warmup, gradient clipping, a token-count-based training schedule, full resume support, JSONL metrics logging, and both single-GPU and multi-GPU (DDP) code paths.
- **Evaluation** (`eval/`): held-out perplexity, a script-mix diagnostic (does the model stay in Urdu?), and a 45-item hand-written few-shot cloze (fill-in-the-blank) benchmark.
- **GPU-run readiness**: a `base.yaml` config and `docs/GPU_RUN.md` describing the one-command launch, a cost estimate, and token-budget reasoning — written but *not yet executed*.

2 The data pipeline (pipeline/)

2.1 Data sources

Two public, no-authentication-required sources are used (details and licenses in `docs/DATA_LICENSES.md`):

Source	Size	License
Urdu Wikipedia dump (article text, namespace 0)	1,728,010 raw articles	CC BY-SA 4.0 / GFDL
Leipzig newscrawl 2016 (1M sentences)	1,000,000 raw sentences	CC BY-NC 4.0
Leipzig newscrawl 2011 (300K sentences)	300,000 raw sentences	CC BY-NC 4.0

Because the Leipzig corpora are non-commercial licensed, any model or artifact trained on them inherits that restriction; the README states that training on Wikipedia alone would avoid this if a permissive corpus were required.

2.2 Reading raw formats (sources.py)

- **Wikipedia:** the raw download is a `.bz2`-compressed MediaWiki XML dump. `read_wiki_dump` streams it with Python’s incremental XML parser (`ET.iterparse`), so the whole (large) dump never has to sit in memory at once. It skips redirect pages and non-article namespaces, and hands each article’s raw *wikitext* (MediaWiki’s markup language, e.g. `[[link]]`, `{{template}}`) to `mwparserfromhell`, a library that strips markup down to plain text (`strip_code`). A small regex mop-up afterwards drops leftover table/template lines that start with `|`, `!`, `{`, or `}`.
- **Leipzig:** the raw download is a `.tar.gz` archive containing a `*-sentences.txt` file, one tab-separated `id\tsentence` pair per line. `read_leipzig_tar` streams this directly from the tar member without ever extracting to disk.

Both readers are Python *generators* (functions using `yield`): they produce one document at a time on demand instead of building a list of everything up front, which is what makes streaming a multi-hundred-megabyte dump on ordinary hardware possible.

2.3 Normalization (normalize.py)

Definition: Unicode normalization (NFC)

The same visible character can have more than one valid digital encoding in Unicode (e.g. a base letter plus a combining diacritic, versus one pre-combined code point). NFC (“Normalization Form Canonical Composition”) rewrites text into one canonical encoding so that visually identical strings become byte-identical, which keeps a tokenizer from treating the same word as two different pieces of vocabulary.

Beyond NFC, the project folds a short list of Arabic/Urdu encoding variants that are the *same letter* but different code points, because several of these are extremely common in scraped Urdu text: Arabic Kaf (U+0643) → Urdu Keheh (U+06A9); Arabic Yeh (U+064A) and Alef Maksura (U+0649) → Farsi Yeh (U+06CC); Heh with hamza above → Heh Goal; and it strips invisible characters (zero-width space, BOM) while *keeping* the zero-width non-joiner, which carries real orthographic meaning in Urdu (it changes

how adjacent letters visually join). It deliberately does *not* strip short-vowel diacritics (*harakat*), since removing them would corrupt the minority of vocalised text (religious quotations, dictionaries) that uses them meaningfully.

Definition: Script / script-range filtering

Every Unicode character lives in a numbered “code point” range, and those ranges are grouped into *blocks* by writing system (e.g. Latin, Arabic, Cyrillic). `is_arabic_char` checks whether a character’s code point falls in one of five Arabic-script Unicode blocks (Arabic, Arabic Supplement, Arabic Extended-A, and the two Arabic Presentation Forms blocks used by some legacy fonts). `arabic_ratio` computes what share of the *alphabetic* characters in a string are in these Arabic-script ranges.

`keep_line` is the boilerplate/language filter applied to every candidate paragraph. It rejects a line if any of these hold:

- fewer than 40 characters, or fewer than 6 words;
- an Arabic-script ratio below 0.65 (i.e. too much non-Urdu text, almost always leaked English);
- fewer than half its characters are alphabetic (catches tables, numeric infobox leftovers).

Design choice: no heavyweight language-ID model

Language identification usually means running a dedicated model (e.g. fastText’s `lid.176`, a 126MB download) that classifies which language a snippet of text is in. This project instead relies on the script-range ratio above. The justification given in the code comments: Urdu shares its Perso-Arabic script with a few other languages, none of which appear in volume in a Wikipedia dump already filtered to `ur` and Leipzig corpora already tagged `urd`; the actual contaminant is embedded English/Latin text, and a script check catches that directly, with no extra dependency or download. This is explicitly called out in the README as something the author would reconsider (add a fastText fallback behind a flag) if ingesting a mixed-script web crawl like CC-100 or OSCAR.

2.4 Exact and near-duplicate removal (`dedup.py`)

Definition: Hash

A function that turns arbitrary data (here, a string of text) into a fixed-size number (a *digest*) such that the same input always produces the same digest, and different inputs almost always produce different digests. Comparing two digests is a cheap stand-in for comparing two full pieces of text.

Exact dedup: every paragraph is hashed with `blake2b` (an 8-byte/64-bit digest) and the digest is stored in a Python `set`. If a paragraph’s hash has been seen before, it is dropped as an exact duplicate.

Definition: MinHash and Locality-Sensitive Hashing (LSH)

Two documents are *near-duplicates* if most of their content overlaps even though they are not byte-identical (e.g. the same sentence with a trailing period added). Checking this exactly for every pair of documents in a multi-million-document corpus is far too slow ($O(n^2)$ comparisons). **MinHash** is a technique that compresses a document into a small fixed-size “signature” such that

the *similarity of two signatures* closely approximates the true similarity (specifically, the *Jaccard similarity*, the size of the set overlap divided by the size of the union) of the two documents' shingle sets. **LSH** (Locality-Sensitive Hashing) buckets similar signatures together so that finding “does anything like this already exist?” is a fast lookup instead of a full pairwise scan.

Definition: Shingle

A short, overlapping substring used as the unit of comparison for near-duplicate detection. This project uses *character 5-grams*: for the text “abcdef” the shingles are abcde, bcdef, etc. (whitespace is removed first). Two texts that share most of their 5-grams are considered near-duplicates.

The **Deduper** class runs both passes in one stream: for each incoming paragraph it first checks the exact-hash set, then builds a MinHash signature and queries the LSH index; if a similar-enough signature is already present (above a configurable Jaccard threshold, default 0.8 in `build_corpus.py`'s CLI, 0.6 in the unit test) the document is dropped as a near-duplicate, otherwise it is kept and inserted into the index.

Challenge from the README: MinHash was the slow stage

The first implementation used `datasketch`'s built-in per-shingle `MinHash.update()` call, which loops in Python and runs a tiny NumPy operation for every single shingle. Benchmarked on real Wikipedia paragraphs this managed only ~250 documents/second, which extrapolated to over two hours for the full corpus. The fix, `build_minhash` in the code shown below, hashes *every shingle of a document at once* with `xxhash` (a fast, deterministic non-cryptographic hash — deterministic across processes, unlike Python's built-in salted `hash()`), then computes the entire MinHash signature with two vectorized NumPy operations instead of a per-shingle Python loop, and writes the result directly into a `datasketch` MinHash object so the LSH banding stays compatible. Measured throughput went to ~2400 docs/s, roughly a 10x speedup, with the same benchmark confirming near-duplicates were still caught correctly.

Listing 1: The vectorized MinHash core (`pipeline/dedup.py`). A and B are the affine hash coefficients shared by every document's signature so LSH comparisons stay valid.

```
def build_minhash(text, num_perm=64):
    m = MinHash(num_perm=num_perm, seed=1,
                permutations=(A, B), scheme="affine32")
    shs = shingles(text) # character 5-grams
    hv = np.fromiter((xxhash.xxh32(s).intdigest() for s in shs),
                    dtype=np.uint64, count=len(shs))
    # affine-permute every shingle hash for every one of the 64
    # permutations in a single vectorized operation:
    phv = (np.outer(hv, A) + B) % MERSENNE
    phv = np.bitwise_and(phv, MAX_HASH)
    m.hashvalues = phv.min(axis=0).astype(np.uint64) # MinHash = elementwise min
    return m
```

Challenge from the README: a library API break

`datasketch` 2.0 changed the MinHash constructor: passing `permutations=` without also passing `scheme=` started raising `ValueError: scheme must be specified explicitly`. The fix was to read the seed MinHash's `scheme` attribute (`"affine32"`) once and pass it explicitly on every con-

structured MinHash, so every document’s signature uses the identical scheme and LSH comparisons remain valid.

2.5 Corpus assembly and split (`build_corpus.py`, `common.py`)

`build_corpus.py` runs three stages — `extract`, `dedup`, `split` — each of which writes a `.stage.done` marker file recording its stats once finished (the `Stage` class in `common.py`). Re-running the command later skips any stage whose marker already exists, unless `-force` is passed; this is what makes the pipeline *resumable* across a crash or a deliberately staged run. Output is written incrementally to disk in numbered shard files (`ShardWriter`), e.g. `clean-00000.jsonl`, `clean-00001.jsonl`, so peak memory usage is bounded by the in-flight hash sets/LSH index rather than by the whole corpus.

The `split` stage assigns each cleaned document to train or validation using a seeded `random.Random` draw against a fixed `val_frac` (1% by default), which makes the split fully reproducible from the same seed.

2.6 Measured corpus statistics (real numbers, from `data/corpus_stats.json`)

Source	Raw units	Kept (post-filter)	Kept after dedup	Chars
Urdu Wikipedia	1,728,010 articles	2,459,293 paragraphs	1,309,831	370.2M
Leipzig newscrawl 2016	1,000,000 sentences	960,189	930,452	122.7M
Leipzig newscrawl 2011	300,000 sentences	276,005	274,799	32.2M
Total		3,695,487	2,515,082 docs	438.5M

Filtering and deduplication together removed **31.9%** of the extracted paragraphs (871,272 exact duplicates + 309,133 MinHash near-duplicates). The README attributes most of this to Urdu Wikipedia’s large population of near-identical, bot-generated stub articles. The final deterministic split is **2,489,976** training documents and **25,106** validation documents (a 1% holdout). After tokenization (next section) this becomes **111.7M** training tokens and **1.1M** validation tokens, **112.8M** total — a number that directly drives the token-budget discussion in Section 9.

3 The tokenizer

Definition: Tokenization / BPE

A neural network cannot consume raw text directly; text is first chopped into a fixed vocabulary of “tokens” (which can be whole words, sub-word pieces, or even single bytes), each mapped to an integer ID. **Byte-Pair Encoding (BPE)** builds that vocabulary by starting from individual bytes and repeatedly merging the most frequent adjacent pair into a new token, up to a target vocabulary size. This lets *common* sequences (whole words, frequent sub-words) become single tokens while *rare* sequences still decompose into smaller, already-known pieces — so no input text can ever be completely unrepresentable (there is no “unknown word” problem the way there is with a fixed word list).

`pipeline/train_tokenizer.py` trains a 32,000-token **byte-level** BPE tokenizer (using Hugging Face’s `tokenizers` library’s algorithm implementation only, not any pretrained model) on the cleaned training shards. “Byte-level” means the starting alphabet is the 256 possible byte values rather than a fixed list of Unicode characters, which guarantees *zero* out-of-vocabulary symbols on any UTF-8 input — this matters specifically for Urdu, whose Perso-Arabic characters are exactly the kind of thing a Latin-centric tokenizer’s fixed character list might not otherwise anticipate. Special tokens reserved in the vocabulary: `<|endoftext|>` (document separator), `<|pad|>`, and `<|unk|>`.

`pipeline/tokenize_corpus.py` then encodes every cleaned JSONL document with this trained tokenizer, appends an `<|endoftext|>` token id after each document, and concatenates every id into one flat binary array saved as `uint16` (a 2-byte unsigned integer, sufficient to index a 32k vocabulary and half the size of a 4-byte `int32`) — `data/train.bin` and `data/val.bin`. Storing token ids as a flat binary file lets training *memory-map* the file (Section 5) instead of loading the whole corpus into RAM.

3.1 Measured compression against GPT-2’s tokenizer

The README reports a direct, measured comparison (`eval/tokenizer_compare.py`) run on an 806,495-character held-out Urdu sample:

Tokenizer	Tokens for the sample	Chars/token	Tokens vs. ours
Ours (32k Urdu BPE)	197,758	4.078	1.0x
GPT-2 (50k English BPE)	1,034,831	0.779	5.23x more

GPT-2’s tokenizer, trained on English web text, ends up emitting close to one token per UTF-8 byte on Urdu (0.78 chars/token) because its learned merges essentially never match Urdu byte sequences. A tokenizer trained natively on Urdu serves the same text for roughly a fifth of the token count, which matters directly for cost: fewer tokens per document means cheaper training and cheaper inference for the same text.

4 The model (model/)

4.1 Overall shape: a pre-norm decoder stack

Definition: Pre-norm

“Norm” here means a *normalization layer*, which rescales a vector of numbers to have a controlled magnitude before it is used, which keeps training numerically stable as networks get deep. “Pre-norm” means the normalization is applied *before* each sub-layer (attention, or the feed-forward network) rather than after; this is the arrangement modern large transformers almost universally use because it keeps gradients better behaved through many stacked layers.

Definition: Residual connection

Rather than replacing a signal entirely, each transformer sub-layer computes a small update and *adds* it back onto the original input ($x \rightarrow x + f(x)$). This “skip connection” means information and gradients have a direct path through the whole network even if an individual sub-layer’s output were zero, which is essential for training very deep networks.

Every block in UrduLM follows the same pattern (Figure 2):

$$x \rightarrow x + \text{Attention}(\text{RMSNorm}(x)) \rightarrow x + \text{SwiGLU}(\text{RMSNorm}(x))$$

4.2 RMSNorm

Definition: RMSNorm (Root Mean Square Normalization)

A simplified, cheaper alternative to the more common LayerNorm. Instead of centering a vector (subtracting its mean) and then scaling it, RMSNorm only rescales each vector by its root-mean-square magnitude: $x \cdot \text{rsqrt}(\text{mean}(x^2) + \epsilon)$, then applies a learned per-channel scale. Skipping the mean-subtraction step is faster and, in practice for transformers, produces results as good as LayerNorm.

The implementation (`RMSNorm` class) explicitly upcasts to 32-bit floating point for the normalization arithmetic and casts back to the input’s original precision afterward, a common trick to avoid numerical instability when the rest of the network runs in a lower-precision format (bfloat16 on GPU, described in Section 5).

4.3 Rotary position embeddings (RoPE)

Definition: Position embedding

Self-attention on its own has no notion of word order — it treats a sequence as an unordered set of positions unless something tells it where each token sits. A *position embedding* injects that information.

Definition: Rotary Position Embedding (RoPE)

Rather than adding a separate position vector to each token (the older approach), RoPE encodes position by *rotating* pairs of dimensions of the query and key vectors by an angle proportional to the token’s position, using a different rotation frequency for each pair of dimensions. Because

it is a rotation, the dot product between a rotated query and a rotated key naturally ends up depending only on their *relative* distance, not their absolute positions, which generalizes better to sequence lengths not seen exactly during training.

`build_rope_cache` precomputes a cosine/sine table for every position up to `max_seq_len`, using a geometric progression of frequencies controlled by `rope_theta` (10,000 by default, the same constant used in the original RoPE paper and most transformer implementations since). `apply_rope` splits each attention head’s vector into two halves, constructs the “rotate by 90 degrees” companion vector $(-x_2, x_1)$, and combines it with the cosine/sine values — exactly the 2D-rotation formula applied independently to each dimension-pair.

4.4 Attention, and grouped-query attention

Definition: Self-attention

For every position in a sequence, self-attention computes a weighted average of *all other positions’* representations (restricted, for a causal/ decoder model, to positions at or before it), where the weights are learned based on how relevant each other position is to the current one. This is what lets the model, e.g., resolve which earlier noun a pronoun refers to.

Definition: Query, Key, Value (Q/K/V)

Self-attention is implemented by projecting each token’s vector into three different vectors: a *query* (“what am I looking for”), a *key* (“what do I contain, for others to match against”), and a *value* (“what do I actually contribute if matched”). The attention weight between two positions is (roughly) the dot product of one position’s query and the other’s key; the output is the weighted sum of values.

Definition: Grouped-query attention (GQA)

A memory/speed optimization: instead of giving every attention head its own key and value projections, several query heads *share* one smaller set of key/value heads. This shrinks the key/value cache used during autoregressive generation with a small accuracy cost.

The `Attention` class projects the input into `n_head` query heads and `n_kv_head` key/value heads (equal by default, so grouped-query attention is available but not enabled in any of the shipped presets — `ModelConfig.n_kv_head` defaults to matching `n_head`). If they differ, `k/v` are repeated (`repeat_interleave`) to match the number of query heads before the attention computation. RoPE is applied to queries and keys, then attention itself is computed with PyTorch’s built-in `scaled_dot_product_attention(..., is_causal=True)` rather than a hand-written attention formula with an explicit mask.

Why `scaled_dot_product_attention`

This single PyTorch call transparently uses a fused, memory-efficient “Flash Attention”-style kernel on supported GPUs and a numerically equivalent fallback elsewhere, and `is_causal=True` generates the causal (no-peeking-at-the-future) mask internally rather than the caller building an explicit upper-triangular mask matrix by hand. The README notes this was deliberately chosen so the model gets a correct, Flash-attention-ready implementation “for free.”

4.5 SwiGLU feed-forward layer

Definition: MLP / feed-forward layer

The second sub-layer in every transformer block: a small two-(or three-)layer neural network applied independently to each position’s vector (it does not mix information across positions the way attention does). This is where most of a transformer’s “knowledge” capacity is generally thought to live.

Definition: SwiGLU

A feed-forward variant using a *gating* mechanism: the input is projected twice, in parallel, into a “gate” vector and an “up” vector; the gate is passed through the SiLU activation function and multiplied element-wise against the up vector before a final projection back down. Empirically, gated variants like SwiGLU tend to outperform a plain single-projection MLP at the same parameter budget.

The hidden dimension is `mlp_ratio × n_embd`, with `mlp_ratio = 8/3 ≈ 2.67` (rounded up to a multiple of 64), rather than the classic GPT-2 ratio of 4. This specific ratio is a known convention (used by e.g. LLaMA) for keeping a *gated* MLP’s total parameter count comparable to a plain 4x-ratio MLP, since SwiGLU has three weight matrices instead of two.

4.6 Tied embeddings

Definition: Tied (weight-shared) embeddings

The layer that turns a token id into a vector at the input (`tok_emb`) and the layer that turns the final hidden vector back into a probability over every vocabulary token at the output (`lm_head`) can, and here do, *share the exact same weight matrix* (`self.lm_head.weight = self.tok_emb.weight`). This roughly halves the parameter cost of the vocabulary-sized weight matrices, and is standard practice in GPT-2-family models.

4.7 Initialization

Weights are initialized from a normal distribution with standard deviation 0.02 (`_init_weights`), a convention carried over from GPT-2. Output projections that feed directly into a residual stream (`o_proj.weight`, `down_proj.weight`) get an extra *scaled* initialization, dividing the standard deviation by $\sqrt{2 \times n_layer}$, which is the same trick GPT-2 used to keep the variance of the residual stream from growing uncontrollably as more layers are stacked.

4.8 Size presets

preset	params	layers	d_model	heads	context
tiny	13.0M	6	256	4	512
small	35.7M	6	512	8	1024
base	123.7M	14	768	12	1024

(`d_model` is the width of every token’s vector as it flows through the network; `context` is the maximum sequence length, i.e. how many tokens of prior text the model can attend to at once.)

Challenge from the README: the tiny preset is embedding-bound

At a 32k vocabulary the tied embedding table alone is 8.2M parameters, so a genuinely small transformer still lands around 13M total, and the project's own parameter-count test initially failed at 15.2M against a tighter target. Rather than shrink the vocabulary to hit a rounder headline number, the block dimensions were tuned to a defensible 13M and the split (8.2M embedding, 4.8M transformer blocks) is reported openly in the README instead of hidden. The **base** preset needed 14 layers, not the more common 12, to reach 124M total parameters, because a SwiGLU block at the 8/3 ratio has fewer parameters per layer than a classic 4x-GELU block.

The unit tests (`tests/test_model.py`, `test_preset_param_counts`) pin acceptable parameter-count bands for all three presets at the real 32k vocabulary size, so a future change to the architecture that silently shifts a preset's size out of its intended band would fail CI.

5 Training (train.py)

5.1 Token-count scheduling, not step-count scheduling

Definition: Epoch / step

A *step* is one optimizer update (one pass of forward + backward + weight update). An *epoch* is one full pass over the training data. Many training scripts are configured by number of steps or epochs; this project is instead configured by a target *token count* (`max_tokens` in the YAML config), converted internally into a step count via `tokens_per_step = batch_size × block_size × grad_accum × world_size`. This makes it easy to reason about training in terms of “how much data has the model actually seen” (directly comparable to the Chinchilla-style token-budget argument in Section 9) rather than in terms of an arbitrary step count that changes meaning whenever the batch size changes.

5.2 Gradient accumulation

Definition: Gradient accumulation

To simulate a larger batch size than fits in memory at once, the model processes several smaller “micro-batches” in a row, adding up (accumulating) their gradients *without* resetting them between micro-batches, and only then performs one optimizer step on the summed gradient. `train.py` divides each micro-batch’s loss by `grad_accum` before calling `.backward()` so that the accumulated gradient’s scale matches what a single true large batch would have produced.

5.3 Mixed precision (AMP)

Definition: Mixed precision / bfloat16 (bf16)

Neural network training is normally done in 32-bit floating point. Modern GPUs compute much faster in a 16-bit floating-point format. `bfloat16` (“brain float 16”) is a 16-bit format that keeps the same exponent range as 32-bit float (so it does not lose the ability to represent very large or very small numbers) while sacrificing precision, which in practice barely affects transformer training quality. `torch.autocast` automatically runs selected operations in bf16 on the fly.

The training loop enables bf16 autocast only when running on a CUDA GPU that supports it (`torch.cuda.is_bf16_supported()`); on CPU it disables autocast entirely and runs in plain 32-bit float, since the tiny proof run described in Section 7 was executed on an 8-core CPU with no GPU acceleration available.

5.4 Cosine learning-rate schedule with warmup

Definition: Learning rate schedule, warmup, cosine decay

The *learning rate* controls how large a step the optimizer takes on each update. *Warmup* means starting the learning rate near zero and ramping it up linearly over the first portion of training, which avoids destabilizing large weight updates before the model has settled at all. A *cosine decay* schedule then smoothly reduces the learning rate from its peak down to a small minimum value following a cosine curve, rather than dropping it in sudden steps.

`cosine_lr(step, warmup, total, base_lr, min_lr)` implements exactly this: linear ramp for `step < warmup`, then a cosine-shaped decay from `base_lr` to `min_lr` over the remaining steps.

5.5 Gradient clipping

Definition: Gradient clipping

Occasionally, a training step produces an unusually large gradient (e.g. from an outlier batch), which can destabilize training with a single bad update. Gradient clipping rescales the gradient if its overall norm exceeds a threshold (here, `grad_clip = 1.0`), leaving normal-sized gradients untouched.

5.6 Resumability

Definition: Checkpoint / resume

A *checkpoint* is a saved snapshot of everything needed to continue training later exactly where it left off. `train.py` saves the model weights, the optimizer's internal state (e.g. Adam's running gradient statistics), the data loader's cursor position, and the step/token counters into `ckpt.pt` at every `ckpt_interval` steps. Passing `-resume` restores all four, so a resumed run produces *the exact same* sequence of batches and optimizer updates as an uninterrupted run would have, not an approximation.

The `TokenLoader` class implements the resumable data cursor: it pre-shuffles a deterministic list of block start-offsets into the memory-mapped token file once (seeded), then walks through that shuffled list with a cursor that wraps back to zero when it reaches the end (i.e. starts a new pass over the data, an "epoch wrap" for a corpus smaller than the token budget — which is exactly what phase 2's `base` run is designed to do, per Section 9). Saving/restoring just the integer cursor value is enough to reproduce the identical batch sequence after a resume.

Two checkpoints are written at the end of training: `ckpt.pt` (full state, for resuming further training) and a separate, smaller `model.pt` containing only the model weights and config (no optimizer state), intended for distribution/inference — this is the file shipped in the repository as `runs/tiny/model.pt` (52MB; the full 156MB optimizer-state checkpoint is not committed).

5.7 Single-GPU and multi-GPU (DDP) paths

Definition: DDP (Distributed Data Parallel)

A technique for training across multiple GPUs (or machines) at once: each GPU holds a full copy of the model and processes a different slice of the batch, then all copies' gradients are averaged (synchronized) before each optimizer step, so every GPU's weights stay identical.

`is_dist()` checks the `WORLD_SIZE` environment variable (set automatically by PyTorch's `torchrun` launcher) to decide whether to initialize a process group and wrap the model in `DistributedDataParallel`. The same `train.py` script therefore runs unmodified as either `python train.py -config ...` (single process) or `torchrun -nproc_per_node=N train.py -config ...` (N-way data parallel), with the `TokenLoader` striding data across ranks (`idxs[rank::world_size]`) so each GPU sees a disjoint slice of every global batch.

6 Evaluation (eval/)

6.1 Held-out perplexity (perplexity.py)

Definition: Perplexity

A standard language-model quality metric derived directly from the cross-entropy loss: $\text{perplexity} = e^{\text{loss}}$. Intuitively, it is the (geometric) average number of equally-likely choices the model was "confused between" when predicting each next token — lower is better. A perplexity of 1 would mean perfect, certain prediction; a perplexity equal to the vocabulary size would mean the model is no better than guessing uniformly at random.

This script loads a checkpoint, memory-maps a held-out `.bin` token file, evaluates the model's loss over held-out blocks, and reports the average loss and its exponential (perplexity).

6.2 Script-mix diagnostic (script_mix.py)

A behavioral sanity check specific to a script-mixing risk: it generates free-form continuations from a handful of Urdu prompts and measures what fraction of the generated *characters* fall in the Arabic-script Unicode ranges (using the same `is_arabic_char` helper from the data pipeline) versus Latin script versus other. A healthy Urdu model should stay almost entirely in Arabic script; a high Latin share would indicate the model is leaking English or degenerating into repetition/garbage.

6.3 Few-shot cloze probe (cloze.py, evaldata/)

Definition: Cloze test

A fill-in-the-blank task: given a sentence with a piece missing (the *stem*) and several candidate completions (*options*, exactly one correct), the task is to pick the correct completion.

Definition: Few-shot

Showing the model a handful of solved examples of a task in its input context *before* asking it to solve a new instance, without any additional training — purely by putting worked examples in the prompt. "Zero-shot" means no such examples are shown.

`evaldata/cloze.jsonl` is a 45-item, hand-authored Urdu cloze set (explicitly *not* drawn from any external benchmark; documented in `evaldata/README.md`), testing basic factual/commonsense knowledge (the capital of Pakistan, water boiling point, etc.) with same-shape grammatical distractors so the task measures meaning rather than surface grammar. For each option, `option_loss` scores the language-model loss of *just the completion tokens* (earlier context tokens are masked out of the loss with `ignore_index=-100`); the option with the lowest loss is taken as the model's answer. `-shots N` optionally prepends N solved examples as a few-shot prefix before scoring the remaining items.

Honesty note, stated directly in the README and evaldata/README.md

This probe is explicitly built and calibrated for the not-yet-trained phase-2 base model. On the tiny 13M/500-step proof-run checkpoint, near-chance accuracy is expected and is reported as such, not as a shortfall.

6.4 Tokenizer compression comparison (`tokenizer_compare.py`)

Covered already in Section ?? (the tokenizer section above); this script is what produced the measured 5.23x figure. If no GPT-2 tokenizer file is supplied, it falls back to comparing against a raw byte-level baseline (one token per UTF-8 byte) instead.

6.5 Loss-curve plotting (`plot_loss.py`)

A small utility that reads `metrics.jsonl` (the JSONL log written by `train.py`) and plots train and validation loss over training steps with `matplotlib`, producing `docs/tiny_run_loss.png` (embedded in Section 7 below).

7 The tiny proof run (executed here, on CPU)

The **tiny** (13M-parameter) preset was trained on an 8-core CPU on a slice of the real corpus, purely to demonstrate that the entire pipeline — data, tokenizer, model, training loop, checkpointing, evaluation — works end to end. Configuration (`configs/tiny.yaml`): block size 256, effective batch size 8,192 tokens/step (batch $16 \times \text{grad_accum } 2$), cosine learning rate schedule peaking at $6\text{e-}4$.

Metric	Value
Steps trained	500 (stopped at a checkpoint boundary)
Tokens seen	4.10M
Train loss	9.74 $\rightarrow$ 5.42
Held-out val loss (during training, 2 evals)	5.87 (ppl 353) $\rightarrow$ 5.36 (ppl 212)
Final held-out perplexity (fuller eval, <code>eval/perplexity.py</code>)	248.8
Script-mix diagnostic (Arabic-script share of output chars)	98.2% (1.8% Latin)
Cloze accuracy (45 items, 3 options, random baseline 0.333)	0.378

What this run does and does not show

500 steps and 13M parameters is a *pipeline proof*, not a capability claim: the README is explicit that sample continuations are "barely coherent by design." The cloze accuracy of 0.378, only slightly above the 0.333 random baseline, is exactly what is expected for this size/step count and is reported as such — the cloze harness was built and calibrated for the not-yet-run phase-2 base model. The loss curve is still visibly, monotonically decreasing at step 500 (not converged), which is the intended point: showing the training loop actually works and reduces loss, not that a small model was pushed to convergence.

The saved slim checkpoint (`runs/tiny/model.pt`, 52MB) contains only model weights and config, not optimizer state. Sample continuations (reproducible with `python sample.py -ckpt runs/tiny/model.pt -seed 7`) are real Urdu words, loosely on-topic (cricket, Pakistan appear repeatedly across prompts) but with loose grammar. The README quotes three full Urdu-script sample continuations verbatim (prompts “*Pakistan a/one*”, “*Urdu language*”, “*today’s weather*”); this PDF is built with `pdflatex`, which cannot correctly shape right-to-left Perso-Arabic script, so rather than render broken glyphs here, the samples are paraphrased in English: the model’s continuations name Pakistan and cricket repeatedly, use real Urdu vocabulary, and stay grammatically loose (e.g. one continuation roughly translates as “Pakistan is a large number who is busy in the Pakistan Cricket League (N)”) — consistent with a 13M-parameter model 500 steps into training. See the top-level `README.md` for the exact Urdu-script transcripts.

8 Testing (tests/)

`python -m pytest tests/ -q` runs **12 tests**, all passing per the README: 5 model tests, 7 pipeline tests.

8.1 Model tests (tests/test_model.py)

- **Forward-pass shapes:** the model’s output logits have the expected (`batch`, `sequence`, `vocab`) shape, and passing targets produces a scalar loss.
- **Causal masking property:** perturbing the *last* input token and re-running the forward pass must leave every *earlier* position’s logits completely unchanged (`torch.allclose`), which directly verifies the causal attention mask is correct without needing to inspect the mask matrix itself — a clean, black-box test of a subtle correctness property.
- **Tied embeddings:** asserts `lm_head.weight` and `tok_emb.weight` are the literal same tensor in memory (`data_ptr()` equality), not just equal in value.
- **Generation runs:** a smoke test that `model.generate` produces the expected output length.
- **Preset parameter counts:** each of `tiny/small/ base` must land within an explicit acceptable parameter-count band at the real 32k vocabulary (the exact bands are given in Section ?? above); this is the test that caught the embedding-dominance issue described earlier.

8.2 Pipeline tests (tests/test_pipeline.py)

- Normalization folds Arabic-encoding variants to their canonical Urdu forms and correctly handles whitespace/invisible characters.
- `arabic_ratio` computes the expected ratio on pure-Urdu, pure-English, and mixed-script text.
- `keep_line` correctly rejects a too-short line and an English sentence, and accepts a genuine Urdu sentence.
- Exact dedup: a repeated document is correctly caught and counted.
- Near dedup: a near-identical document (differing only by a trailing punctuation mark) is correctly caught by the MinHash/LSH machinery.
- Hash stability: the same input to `exact_key` always produces the same digest.

A test caught a real authoring bug, per the README

While writing the `arabic_ratio` test, the author’s own initial expectation was wrong: for the mixed string “(Urdu word for ‘Urdu’) and english”, the Arabic-script ratio is 4 Arabic letters out of 16 total letters = 0.25, not the 0.5 the author first assumed. The test now pins the correct value (asserting it lies strictly between 0.2 and 0.4), and the README cites this directly as an example of why writing the filter’s test surfaced an off-by-count bug in the author’s own reasoning, not the code.

9 Status: phase 1 done, phase 2 not yet run

9.1 What is actually done

Phase 1 (this repository, as of the current commit): done and verified. The real corpus described in Section 2 was built, the tokenizer was trained, all 12 tests pass, and the `tiny` model was trained on CPU with a demonstrably decreasing loss curve and a saved, reproducible checkpoint.

Phase 2 (the ~124M base model on a GPU): not trained yet. The README is explicit that this is "a budget decision, not an engineering one" — everything needed to launch it is written and configured (`configs/base.yaml`, `docs/GPU_RUN.md`), but the run itself has not been executed, and **no base-model weights, checkpoints, or evaluation numbers exist in this repository.** Any number that appears elsewhere in this document under "tiny proof run" belongs *only* to the 13M CPU model, never to `base`.

9.2 The token-budget argument (Chinchilla-style), as written in `docs/GPU_RUN.md`

Definition: Chinchilla scaling / compute-optimal training

A widely cited empirical finding (DeepMind's "Chinchilla" paper) is that, for a fixed compute budget, there is a roughly optimal balance between model size and the number of training tokens: very roughly, about 20 training tokens per model parameter. Training a model on far fewer tokens than this ratio suggests leaves potential quality on the table; training on far more than the available *distinct* data supports risks diminishing returns or overfitting through repetition.

For a 124M-parameter model, the 20-tokens-per-parameter rule points to about **2.5B** training tokens. The cleaned corpus actually available is **111.7M** training tokens (112.8M including validation) — about **0.9 tokens per parameter**, or roughly **4.5%** of the compute-optimal budget. `docs/GPU_RUN.md` states this plainly: the `base` run, as planned, is **data-limited, not compute-limited**.

Reaching the full 2.5B-token Chinchilla target would mean repeating the existing corpus roughly 22 times, which the document judges would overfit badly (citing Muennighoff et al., "Scaling Data-Constrained Language Models," 2023, for the more modest finding that repeating a corpus up to about 4 epochs degrades little). The documented, defensible plan instead:

1. Train for 2–4 epochs (~220M–450M tokens) — this is exactly what `configs/base.yaml` is set to (`max_tokens: 450000000`), expecting "a capable-for-its-data model, not a compute-optimal one."
2. Grow the corpus before spending more GPU money on more epochs of the same data — e.g. CC-100 (Urdu) alone would roughly 10–20x the token count. This is called out as the actual highest-leverage next step, not more compute on the current data.

9.3 Cost estimate for the planned run (documented, not yet spent)

`docs/GPU_RUN.md` quotes RunPod Community Cloud on-demand prices (checked 2026-07-06) and a conservative throughput estimate (~30k tokens/second on a single A100 80GB, bf16 + Flash attention via `scaled_dot_product_attention`) to estimate the planned 450M-token run (`configs/base.yaml`) at roughly **4.2 hours** wall-clock and about **\$6**. The document is explicit that this throughput number is an *estimate*, not a measurement — "the only measured throughput in this repo is the CPU proof run."

Real GPU session log, but no training completed yet

`docs/GPU_RUN.md` also documents a real RunPod session on an A100 SXM 80GB (2026-07-06) covering practical operational details: SSH key setup quirks on RunPod, transferring the pre-built `data//tokenizer/` directories via `rsync` rather than rebuilding on the pod, avoiding accidentally overwriting RunPod's pre-installed CUDA-enabled torch build with a CPU-only wheel from `requirements.txt`, running training detached under `tmux` so a multi-hour run survives an SSH disconnect, and a note that stopping RunPod's billing meter requires terminating the pod from the web dashboard (SSH access cannot do this). This documents *operational readiness*, not a completed training run; no resulting checkpoint, loss curve, or evaluation number from a GPU run appears anywhere in the repository.

10 What the author learned and would do differently

Both lists below are transcribed directly from the README's "What I learned" and "What I'd do differently" sections, included because they are part of how the author would explain this project's engineering process:

- Vectorizing the actual hot loop mattered more than picking a "fast" library — the same `datasketch` LSH went from unusable to fine purely by moving per-element work into two NumPy calls per document; profile the real workload before scaling it up, not after.
- MinHash signatures only need to be internally consistent for LSH to work, which is what allowed swapping `datasketch`'s default sha1 element hash for xxhash and setting `hashvalues` directly, as long as every document used the same scheme and permutations.
- With a real 32k subword vocabulary, a "10M-parameter" model is a fiction below a certain size, since the embedding table dominates; reporting the embedding-vs-block split honestly is better than tuning the vocabulary size to hit a headline number.
- SwiGLU at an 8/3 hidden ratio is deliberately sized to match a 4x-GELU block's parameter count, so a SwiGLU model needs more depth than a classic GPT-2 to reach the same total parameter count.
- RoPE plus `scaled_dot_product_attention(is_causal=True)` is enough to get a correct, Flash-attention-ready causal transformer with no hand-written mask, and the causal property is cheap to test directly.
- A resumable, token-count-scheduled training loop is worth building up front: it is the difference between "the phase-2 GPU run is one command" and "someone has to babysit step counts."
- **Parallelize extraction** — the single-process wikitext parse is the slowest stage by far; a multiprocessing pool over pages, or pre-extracted Wikipedia datasets, would cut corpus build time several-fold.
- **Persist MinHash signatures to disk** — dedup currently holds the LSH index only in RAM and recomputes signatures on every run; persisting them would make dedup independently resumable and allow re-tuning the Jaccard threshold without re-hashing everything.
- **Cap Wikipedia bot stubs earlier** — a source-specific pre-filter on templated/short pages would shrink input to the dedup stage rather than relying on dedup to clean it up afterward.
- **Add a real fastText LID fallback as an option** — the script-range heuristic is right for the current sources, but a mixed-script web crawl (CC-100, OSCAR) would need proper token-level language identification, wired in behind a flag.

- **Tokenizer ablations** — only one 32k byte-level BPE was shipped; measuring 24k/48k vocabularies and a Unigram model against the same Urdu sample would be worth doing before committing the phase-2 vocabulary.

11 License

Code: MIT License, Copyright (c) 2026 Salman Adnan ([LICENSE](#)). Training data carries its own separate terms ([docs/DATA_LICENSES.md](#)): Urdu Wikipedia is CC BY-SA 4.0, and the Leipzig corpora are CC BY-NC 4.0 — so any released weights trained on the Leipzig portion of the corpus inherit a non-commercial restriction.

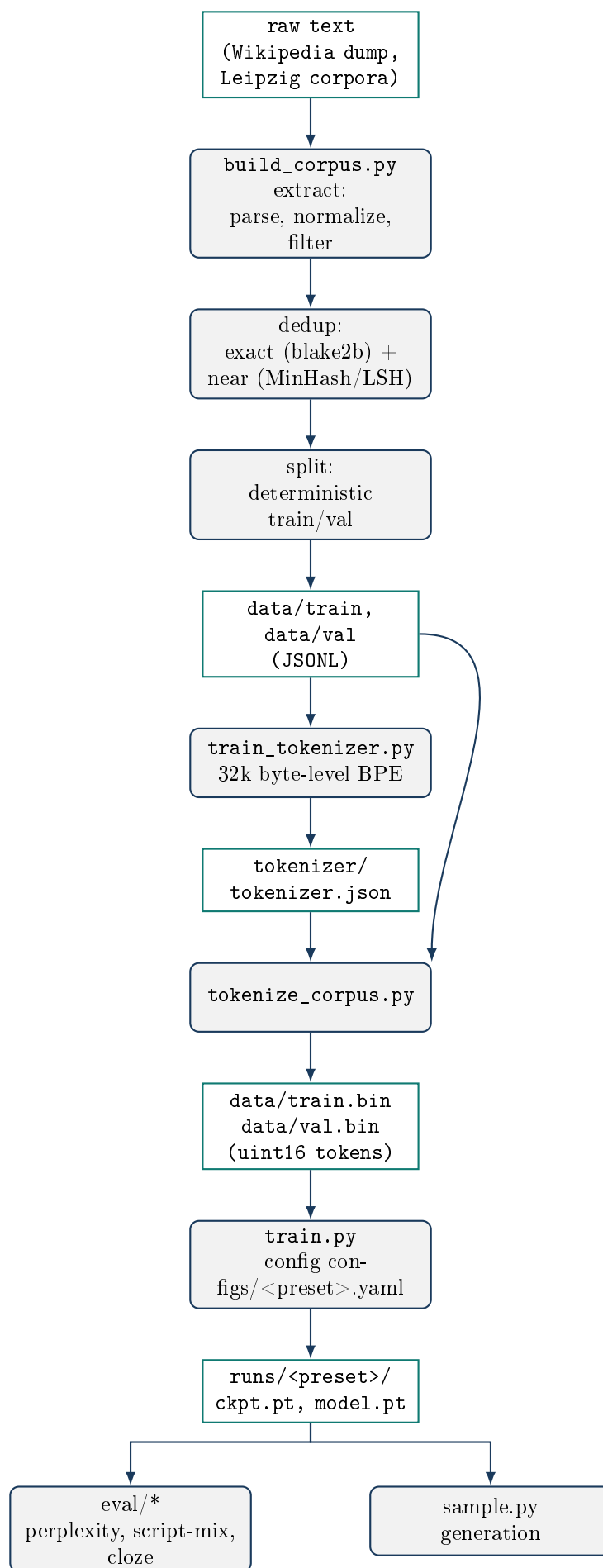


Figure 1: The full data to model pipeline. Every box is an actual file in the repository; arrows are

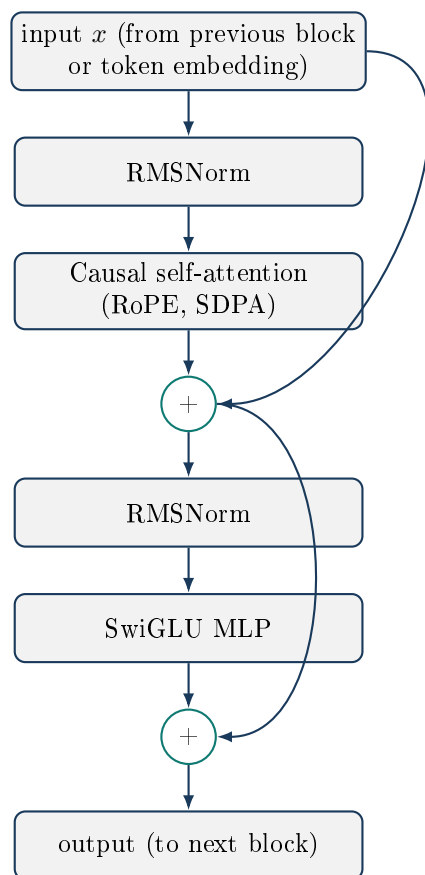


Figure 2: One transformer block (`model/transformer.py`, class `Block`). The curved arrows are the two residual connections. This block is stacked `n_layer` times (6, 6, or 14 depending on preset).

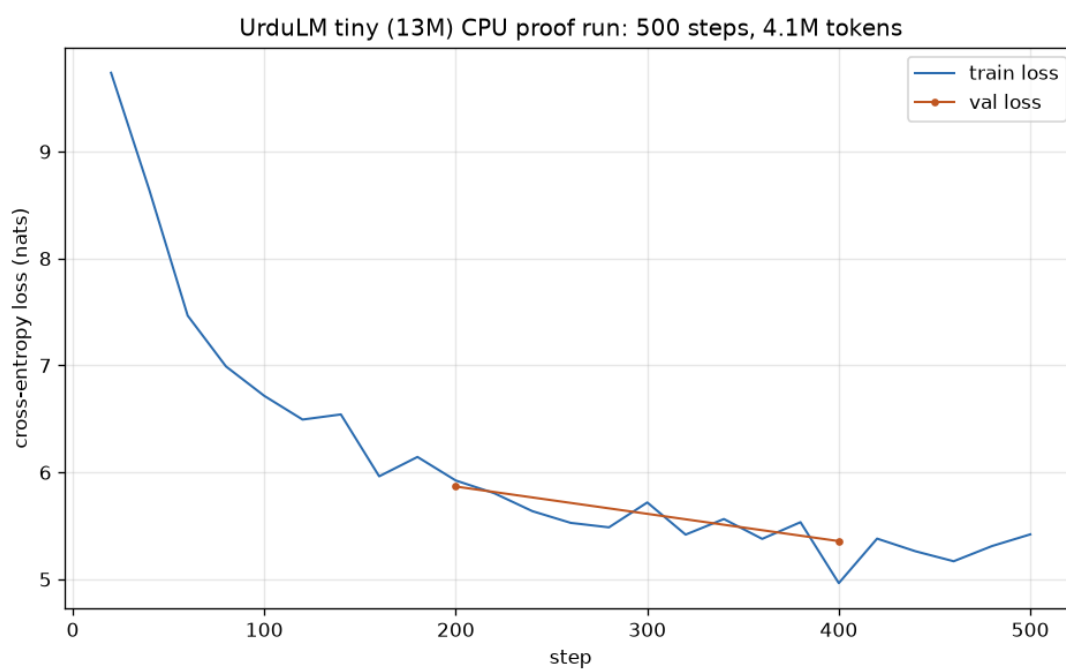


Figure 3: Train/validation cross-entropy loss over the 500-step CPU proof run (docs/tiny_run_loss.png, produced by eval/plot_loss.py).